LANGCHAIN 1.4+ • AGENTIC LIFECYCLE & GOVERNANCE

Agent Middleware & Execution Governance

Mastering SummarizationMiddleware (Message & Token Triggers), HumanInTheLoopMiddleware (Approve, Edit, Reject), and Checkpointed Resilience

An authoritative technical deep dive into agent lifecycle interception and governance. Deconstructs how enterprise AI agents bound context consumption using automated sliding-window summarizers, halt high-stakes tool execution via Human-In-The-Loop approval gates, and maintain persistent state checkpoints.

Target Notebook: `6-middleware.ipynb`

Generation Date: October 26, 2026

Language & Runtime: Python 3.14 • Virtualenv (.venv)

Core Framework: LangChain 1.4.7 • LangGraph 0.3+ • Groq

B. Learning Objectives

After studying this notebook and study guide, you will master:

- **The Agent Interceptor Architecture:** Understand how LangChain 1.4 wraps compiled state graphs with middleware to inject pre-model, pre-tool, and post-execution hooks without altering business logic.
- **Checkpointer & Thread-State Binding:** Configure `InMemorySaver` and understand why persistent session state via `thread_id` is mandatory for pausing and resuming agents.
- **Message-Count Summarization:** Configure `SummarizationMiddleware(trigger=("messages", N), keep=("messages", K))` to automatically compact long conversation histories while preserving the latest turns.
- **Token-Budget Summarization:** Enforce strict token consumption ceilings (`trigger=("tokens", 550), keep=("tokens", 200)`) to prevent context exhaustion, rate limits, and latency spikes across multi-turn tool loops.
- **Human-In-The-Loop Governance:** Intercept dangerous or sensitive tool invocations (such as sending emails or modifying databases) before execution using `HumanInTheLoopMiddleware` and `interrupt_on` .
- **The 3-Way Human Review Protocol:** Master the `langgraph.types.Command(resume=...)` protocol to dynamically **Approve**, **Reject**, or **Edit** tool parameters before dispatching actions.

C. Prerequisites & Setup

> **Environment Specification**
> This notebook runs against the high-throughput **Groq LPU Inference Cloud** executing the `openai/gpt-oss-20b` reasoning model (131,072 input context window, native tool-calling and reasoning traces enabled).

Component	Configuration / Package	Purpose
Python Runtime	`Python >= 3.14`	Modern runtime supporting advanced type hinting, dictionary unions, and high-performance execution.
Virtual Environment	`.venv` (managed via `uv`)	Strictly isolates dependencies defined in `pyproject.toml` .
State Checkpointer	`langgraph.checkpoint.memory.InMemorySaver`	Thread-isolated in-memory storage required to suspend and resume graphs during human reviews.
Core Dependencies	`langchain>=1.4.7` , `langchain-groq` , `langgraph` , `python-dotenv`	Supplies universal agent constructors, middleware abstractions, and Groq inference drivers.

> **Security & Safety Rule**
> Never allow autonomous models unsupervised access to external side-effecting APIs (e.g., payment gateways, email dispatches, file deletion). In production, sensitive functions must always be guarded by `HumanInTheLoopMiddleware` .

D. Topic Overview: The Architecture of Agent Middleware & Governance

In raw LLM pipelines, autonomous agents loop without friction: the model decides on a tool, the runtime executes it, and the observation is immediately fed back into the context. While powerful, this unconstrained autonomy introduces two major enterprise vulnerabilities:

1. **Context Window Blowout & Cost Explosion:** Long-running conversational agents or multi-step researchers rapidly accumulate messages and tool outputs. Without proactive memory pruning, token counts grow linearly, quickly triggering provider rate limits, ballooning API costs, and causing latency degradation.
2. **Uncontrolled Side Effects & Hallucinated Actions:** If an autonomous agent decides to execute a destructive or public action (e.g., emailing a customer, deleting database records, executing financial trades), standard autonomous loops dispatch the call immediately without human oversight.

LangChain 1.4 addresses these challenges through the **Agent Middleware System** built atop LangGraph's `CompiledStateGraph`. Middleware acts as an interceptor pipeline surrounding the model decision node and tool execution nodes:

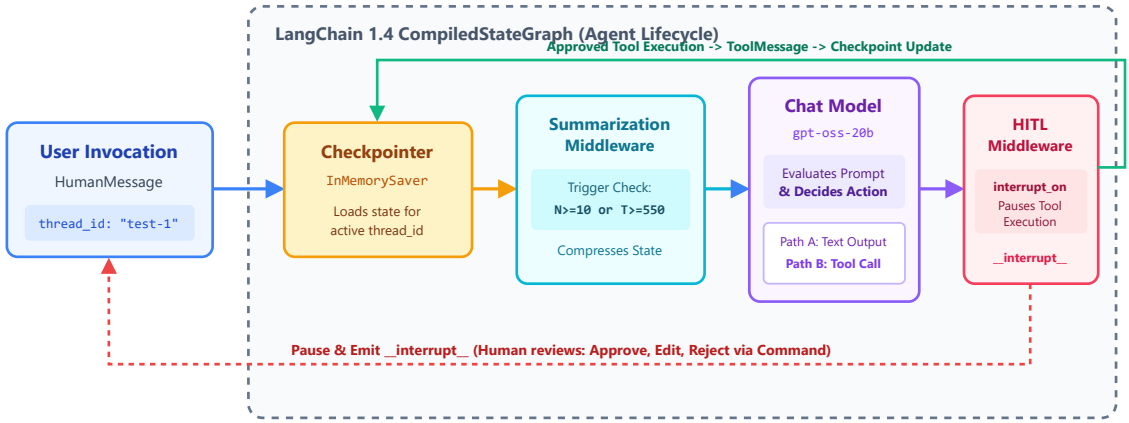


Figure 6.1: LangChain 1.4 Agent Lifecycle Interceptor Pipeline with Checkpointer, Summarization, and HITL Gates.

As illustrated above, every incoming message is assigned a `thread_id` managed by a `checkpointer`. Before the model is called, **SummarizationMiddleware** checks memory thresholds and compacts history if required. When the model emits a tool call, **HumanInTheLoopMiddleware** checks whether the targeted tool requires approval. If intercepted, the entire state machine suspends execution, yields an `__interrupt__` payload to the client, and waits for a signed human `Command` before proceeding.

E. Detailed Notebook Walkthrough & Cell Dissection

Cell 0: Markdown Section Header

```
### Middleware
```

Analysis: Establishes the study module exploring agent middleware interceptors.

Cell 1: Environment & Secret Initialization

```
import os
from dotenv import load_dotenv

load_dotenv()
os.environ["GROQ_API_KEY"] = os.getenv("GROQ_API_KEY")
```

Deconstruction: Loads local environment variables from `.env` and configures the `GROQ_API_KEY` credential in the process environment. This ensures secure API communication without hardcoding keys.

Cell 2: Markdown Section Header

```
### Summrization Middleware
```

Analysis: Initiates the analysis of automated context compression middleware.

Cell 3: Imports & Model Profile Inspection

```
from langchain.chat_models import init_chat_model
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_core.tools import tool

model = init_chat_model("groq:openai/gpt-oss-20b")
model
```

```
EXECUTION OUTPUT (MODEL PROFILE)

ChatGroq(metadata={'lc_versions': {'langchain-core': '1.7.0', 'langchain': '1.4.7'}}, profile={'name': 'GPT OSS 20B',
'release_date': '2025-08-05', 'open_weights': True, 'max_input_tokens': 131072, 'max_output_tokens': 65536, 'text_inputs':
True, 'reasoning_output': True, 'tool_calling': True, 'structured_output': True}, ...)
```

Architectural Rationale:

- `init_chat_model("groq:openai/gpt-oss-20b")`: Initializes Groq's high-throughput open-weights reasoning model.
- `InMemorySaver()`: An essential component. Middleware that pauses execution or manages persistent thread state requires a state saver. `InMemorySaver` stores thread checkpoints in RAM.

Cell 4: Message-Length Based Summarization

```
# Message Length Based Summarization
agent = create_agent(
    model=model,
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(
            model=model,
            trigger=("messages", 10), # summarize after 10 messages
            keep=("messages", 4)
        )
    ]
)

config = {"configurable": {"thread_id": "test-1"}}

questions = [
    "What is 2+2?",
    "What is 10*5?",
    "What is 100/4?",
    "What is 15-7?",
    "What is 3*3?",
    "What is 4*4?",
]

for question in questions:
    response = agent.invoke({"messages": [HumanMessage(content=question)]}, config)
    print(f"Messages: {len(response['messages'])}")
```

```
EXECUTION OUTPUT (MESSAGE COUNT TRACING)

Messages: 2
Messages: 4
Messages: 6
Messages: 8
Messages: 10
Messages: 6
```

Detailed Analysis & Compaction Mechanics:

1. **Threshold Accumulation (Turns 1 to 5):** Each user query produces 1 `HumanMessage` and 1 `AIMessage`. The conversation grows steadily: 2 $ ightarrow$ 4 $ ightarrow$ 6 $ ightarrow$ 8 $ ightarrow$ 10 messages.
2. **Compaction Trigger Firing (Turn 6):** On the 6th question (`"What is 4*4?"`), the middleware detects that message count has reached the trigger threshold of `10` .
3. **Structured Background Summarization:** The middleware invokes the background summarizer model with a specialized prompt to extract:
    - `## SESSION INTENT` : Solves arithmetic queries.
    - `## SUMMARY` : Compact bullet points of questions 1 through 4 and their results.
    - `## ARTIFACTS` : Any code or data created.
    - `## NEXT STEPS` : Current pending question to answer.
4. **State Pruning:** The oldest 6 messages are replaced by a single synthetic `HumanMessage` tagged with `additional_kwargs={'lc_source': 'summarization'}` .
5. **Retention of Recent History:** Because `keep=("messages", 4)` was specified, the 4 most recent messages (Turn 4 & Turn 5 Q&A) are preserved intact alongside the summary. Adding the new Turn 6 query and response yields: $1 ext{ (Summary)} + 4 ext{ (Retained)} + 1 ext{ (New Turn AI)} = 6 ext{ messages}$!

Cell 5: Token-Size Based Summarization

```
# Token Size Summarization
@tool
def search_hostels(city: str) -> str:
    """Search hotels - returns long response to use more tokens."""
    return f"""Hotels in {city}:
    1. Grand Hotel - 5 star, $350/night, spa, pool, gym
    2. City Inn - 4 star, $180/night, business center
    3. Budget Stay - 3 star, $75/night, free wifi"""

agent = create_agent(
    model=model,
    tools=[search_hostels],
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(
            model=model,
            trigger=("tokens", 550),
            keep=("tokens", 200)
        )
    ]
)

config = {"configurable": {"thread_id": "test-1"}}

def count_tokens(messages):
    total_chars = sum(len(str(message.content)) for message in messages)
    return total_chars // 4

cities = ["Paris", "London", "Tokyo", "New York", "Dubai", "Singapore"]

for city in cities:
    response = agent.invoke({"messages": [HumanMessage(content=f"Find hotels in {city}")]}, config=config)
    tokens = count_tokens(response["messages"])
    print(f"{city}: ~{tokens} tokens, {len(response['messages'])} messages")
```

```
EXECUTION OUTPUT (TOKEN STABILIZATION TRACING)

Paris: ~210 tokens, 4 messages
London: ~409 tokens, 8 messages
Tokyo: ~459 tokens, 5 messages
New York: ~499 tokens, 5 messages
Dubai: ~497 tokens, 5 messages
Singapore: ~512 tokens, 5 messages
```

Deconstruction & Token Budget Mechanics:

- `count_tokens(messages)`: Implements the industry heuristic of $\sim 4 ext{ characters} \approx 1 ext{ token}$.
- **Turn 1 & Turn 2:** Paris generates 4 messages (User, AI tool_call, Tool result, AI formatted table) totaling 210 tokens. London appends another 4 messages, raising the token footprint to 409 tokens (8 messages).
- **Turn 3 (Tokyo - Trigger Met):** Ingesting Tokyo would push the context past the 550 token ceiling. The middleware intercepts, invokes the summarizer, collapses Paris and London into a structured summary markdown, retains ~200 tokens of recent context, and returns a compacted 5-message state with only 459 tokens!
- **Continuous Token Bounding:** Across New York, Dubai, and Singapore, the token count stabilizes in a tight band around ~497–512 tokens. The context never runs away linearly, ensuring predictable latency and zero token overflow errors.

Cell 6: Markdown Section Header

```
### Human In The Loop Middleware
```

Analysis: Introduces human approval gates and interactive execution control.

Cell 7: Intercepting High-Stakes Tools via interrupt_on

```
from langchain.agents.middleware import HumanInTheLoopMiddleware

def read_email_tool(email: str) -> str:
    """Mock Function to read an email by its id"""
    return f"Email Content for ID: {email}"

def send_email_tool(recipient: str, subject: str, body: str) -> str:
    """Mock function to send an email"""
    return f"Email sent to {recipient} with subject: {subject}"

agent = create_agent(
    model=model,
    tools=[read_email_tool, send_email_tool],
    checkpointer=InMemorySaver(),
    middleware=[HumanInTheLoopMiddleware(
        interrupt_on={
            "send_email_tool": {
                "allowed_decisions": ["approve", "edit", "reject"]
            },
            "read_email_tool": False,
        }
    )]
)

config = {"configurable": {"thread_id": "test-approve"}}

email = "varad0404@gmail.com"
subject = "greetings"
body = "how are you"

result = agent.invoke(
    {"messages": [HumanMessage(content=f"send email to {email}, with subject: {subject} containing text: {bod
y}")]},
    config=config
)
result
```

```
EXECUTION OUTPUT (STATE SUSPENSION & __INTERRUPT__)

{'messages': [HumanMessage(content='send email to varad0404@gmail.com...'),
              AIMessage(content='', tool_calls=[{'name': 'send_email_tool', 'args': {'body': 'how are you', 'recipient':
'varad0404@gmail.com', 'subject': 'greetings'}, 'id': 'fc_36d...'}])],
 '__interrupt__': [Interrupt(value={'action_requests': [{'name': 'send_email_tool',
                                                         'args': {'body': 'how are you', 'recipient':
'varad0404@gmail.com', 'subject': 'greetings'},
                                                         'description': "Tool execution requires approval\n\nTool:
send_email_tool\nArgs: {'body': 'how are you', 'recipient': 'varad0404@gmail.com', 'subject': 'greetings'}"}],
                                    'review_configs': [{'action_name': 'send_email_tool', 'allowed_decisions': ['approve',
'edit', 'reject']}]},
                            id='cac6e32465a2b8d14fc3f5bc59fb0358', response_schema=None)]}
```

Critical Architectural Highlights:

1. **Selective Interception:** `read_email_tool: False` executes automatically without pausing because reading data carries low operational risk. In contrast, `send_email_tool` is guarded with `allowed_decisions: ["approve", "edit", "reject"]`.
2. **Halted Execution:** Notice that `send_email_tool` was *not* executed. No `ToolMessage` exists in the message list.

3. **The `__interrupt__` Structure:** The graph yields an `Interrupt` object containing the exact proposed arguments (`action_requests`) and the approved decision schema. The current graph state is safely serialized into the `InMemorySaver` under `test-approve`.

Cell 8: Decision Path A - Human Approval (approve)

```
# approve
from langgraph.types import Command

if "__interrupt__" in result:
    print("PAUSED waiting for approval...")

    result = agent.invoke(
        Command(
            resume={
                "decisions": [
                    {"type": "approve"}
                ]
            }
        ),
        config=config
    )

    print(f"Result: {result['messages'][-1].content}")
```

```
EXECUTION RESULT

PAUSED waiting for approval...
Result: ✅ Your email has been sent to **varad0404@gmail.com** with the subject **“greetings”** and the message **“how are you.”**
```

Mechanics: Passing `Command(resume={"decisions": [{"type": "approve"}]})` informs the state machine that the human operator authorized the action. The graph resumes from the exact paused node, invokes `send_email_tool` with the original arguments, collects the tool result, and prompts the LLM to formulate the final confirmation response.

Cell 9: Decision Path B - Human Rejection (reject)

```
# reject
# (Agent initialized with same config & prompt)
result = agent.invoke(
    {"messages": [HumanMessage(content=f"send email to {email}, with subject: {subject} containing text: {bod
y}")]},
    config=config
)

if "__interrupt__" in result:
    print("PAUSED waiting for approval...")

    result = agent.invoke(
        Command(
            resume={
                "decisions": [
                    {"type": "reject"}
                ]
            }
        ),
        config=config
    )

    print(f"Result: {result['messages'][-1].content}")
```

```
EXECUTION RESULT

PAUSED waiting for approval...
Result: I'm sorry, but I can't send the email directly right now.
Would you like me to draft the email for you so you can copy it into your own mail client?
```

Mechanics: When the human supplies `{"type": "reject"}`, the middleware cancels the tool execution. The underlying Python function is never called. A rejection notification is fed into the conversation state, allowing the LLM to understand that permission was denied and gracefully pivot to an alternative solution (offering to draft text instead of sending).

Cell 10: Decision Path C - Human Modification (edit)

```
# edit
# (Agent initialized with same configuration)
correct_email = "varad0404@gmail.com"
wrong_email = "vrad0404@gmail.com"
subject = "greetings"
body = "how are you"

# Prompt contains typo in recipient
result = agent.invoke(
    {"messages": [HumanMessage(content=f"send email to {wrong_email}, with subject: {subject} containing text:
{body}")]},
    config=config
)

if "__interrupt__" in result:
    print("PAUSED waiting for approval...")

    result = agent.invoke(
        Command(
            resume={
                "decisions": [
                    {
                        "type": "edit",
                        "edited_action": {
                            "name": "send_email_tool",
                            "args": {
                                "recipient": correct_email,
                                "subject": subject,
                                "body": body
                            }
                        }
                    }
                ]
            }
        ),
        config=config
    )

    print(f"Result: {result['messages'][-1].content}")
```

```
EXECUTION RESULT

PAUSED waiting for approval...
Result: ✅ Email sent to varad0404@gmail.com with subject “greetings” and body “how are you”.
```

Mechanics: The human detects a critical typo in the recipient's email address (`vrad0404@...`). Instead of rejecting the entire run or starting over, the operator provides `edited_action` containing the corrected parameters. The middleware injects the modified dictionary into the tool dispatcher, executing the function with `varad0404@gmail.com` without re-prompting the LLM.

F. End-to-End Execution Flow & Dedicated Dataflow Diagrams

The notebook highlights two distinct middleware paradigms: **Autonomous Memory Optimization** (Summarization) and **Human-Supervised Execution Governance** (HITL). Below are the explicit dataflow diagrams capturing the internal mechanics of each middleware.

1. SummarizationMiddleware Dataflow (Message Count & Token Budgeting)

When conversation length or token consumption crosses configured thresholds, the middleware interrupts the message append pipeline, invokes a dedicated summarizer model, and injects a single structured recap.

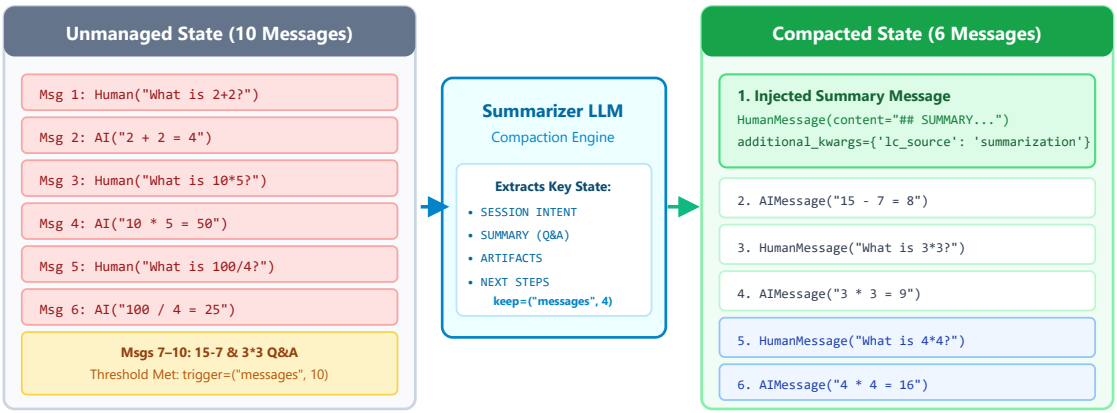


Figure 6.2: State Compaction Cycle: 10 Accumulated Messages Compressed into 1 Summary Message + 4 Preserved Turns.

2. HumanInTheLoopMiddleware 3-Way Decision Dataflow

When an agent targets an intercepted tool, execution is suspended, allowing external operators to evaluate arguments and select one of three resolution paths.

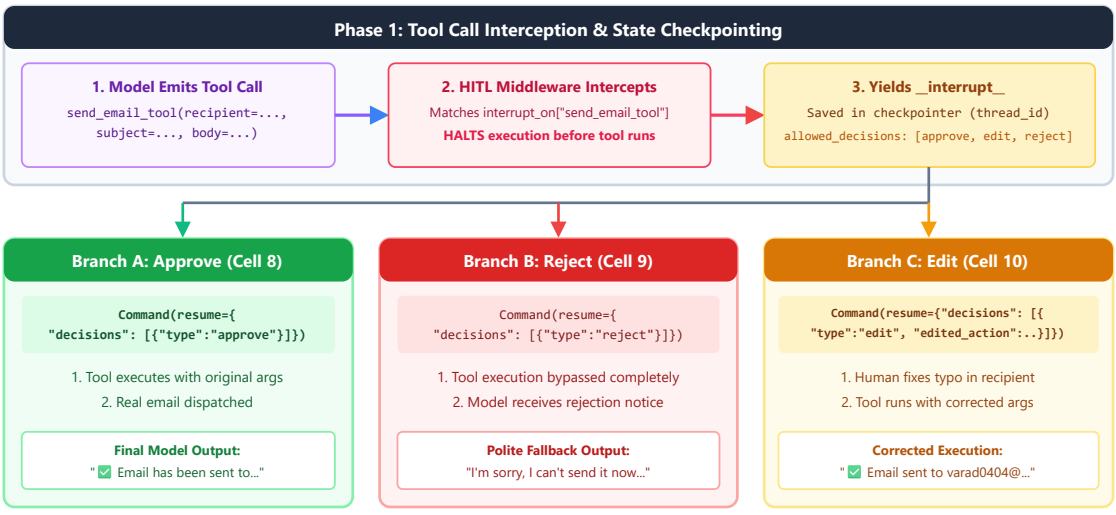


Figure 6.3: The 3-Way HITL Review Protocol: Interception, State Checkpointing, and Branching Resumption (Approve, Reject, Edit).

G. Reference Guide: Key Classes, Functions & Configurations

Class / Function	Import Path	Key Parameters & Configurations	Runtime Role & Behavior
`SummarizationMiddleware`	`langchain.agents.middleware`	`model`, `trigger=("messages"|"tokens", N)`, `keep=("messages"|"tokens", K)`	Lifecycle hook that monitors message history and replaces older turns with a structured summary when thresholds are met.
`HumanInTheLoopMiddleware`	`langchain.agents.middleware`	`interrupt_on: dict[str, dict | bool]`, `allowed_decisions: list[str]`	Intercepts designated tool calls, halts graph execution, and emits an `Interrupt` payload for human review.
`InMemorySaver`	`langgraph.checkpoint.memory`	None (in-memory dictionary backend)	Checkpointer required to serialize, persist, and resume graph execution states across thread boundaries.
`Command`	`langgraph.types`	`resume: dict`, `update: dict (optional)`	Control signal used to unpause an interrupted graph and pass back review decisions (approve, reject, edit).
`Interrupt`	`langgraph.types`	`value: dict`, `id: str`	Payload yielded in `response['__interrupt__']` describing the pending tool call and available decision options.

H. Common Errors, Debugging & Troubleshooting Guide

1. Missing Checkpointer When Using Interrupts or Memory

Symptom: `RuntimeError: State machine requires a checkpointer to support interrupt() or resume().`

Cause: Attempting to initialize `create_agent(middleware=[HumanInTheLoopMiddleware(...)])` without supplying `checkpointer=InMemorySaver()`.

Fix: Always pass a checkpointer instance. LangGraph cannot pause execution or resume without a state storage layer.

2. Invoking an Interrupted Agent Without thread_id

Symptom: `ValueError: No thread_id provided in configuration. Cannot locate checkpoint.`

Cause: Calling `agent.invoke(...)` without `config={"configurable": {"thread_id": "session_abc"}}`.

Fix: All stateful sessions and interrupted agents require an explicit `thread_id` so the checkpointer can index and retrieve the paused node.

3. Malformed Command.resume Payload Structure

Symptom: `ValidationError: Expected 'decisions' list with valid 'type' key.`

Cause: Passing raw strings like `Command(resume="approve")` instead of the expected schema: `Command(resume={"decisions": [{"type": "approve"}]})`.

Fix: Always format the resume dictionary to match the `allowed_decisions` specified in `review_configs`.

I. Practical Extensions & Production Design Patterns

Pattern: Unified Production Agent with Persistent Database Checkpointing

In real-world enterprise deployments, in-memory checkpointers lose all state if the application restarts. The code below demonstrates how to combine both **Summarization** and **HITL** with production-grade persistence and domain-specific approval filtering:

```
import os
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware, HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver # In production: from langgraph.checkpoint.postgres impor
t PostgresSaver

# 1. Initialize models and checkpointer
# In production: checkpointer = PostgresSaver.from_conn_string(os.getenv("DATABASE_URL"))
checkpointer = InMemorySaver()

# 2. Define tools
def execute_sql_query(query: str) -> str:
    """Execute SQL against reporting database."""
    return f"Executed query: {query}"

def export_csv_report(filename: str) -> str:
    """Export generated analytics data to CSV."""
    return f"Saved report to {filename}"

# 3. Create agent with multi-middleware governance
governed_agent = create_agent(
    model=model,
    tools=[execute_sql_query, export_csv_report],
    checkpointer=checkpointer,
    middleware=[
        # A. Protect against token blowout
        SummarizationMiddleware(
            model=model,
            trigger=("tokens", 2000),
            keep=("tokens", 800)
        ),
        # B. Guard destructive SQL queries
        HumanInTheLoopMiddleware(
            interrupt_on={
                "execute_sql_query": {
                    "allowed_decisions": ["approve", "edit", "reject"]
                },
                "export_csv_report": False # Read-only action runs freely
            }
        )
    ]
)
```

J. Key Takeaways & Conceptual Map

- **Middleware Decouples Concerns:** Middleware allows developers to manage memory limits, auditing, and safety gates outside of the core prompt engineering workflow.
- **Checkpointers Enable State Resumption:** Any agent workflow that requires pauses, reviews, or multi-turn conversational history must be bound to a checkpointer with an explicit `thread_id`.

- **Summarization Prevents Runaway Costs:** `SummarizationMiddleware` converts linear token accumulation into a bounded, predictable memory ceiling via structured LLM recaps.
- **HITL Gives Humans Final Authority:** The 3-way resume protocol (Approve, Reject, Edit) guarantees that autonomous agents never execute sensitive real-world side effects without human verification.

K. Practice Exercises & Knowledge Checks

Question 1 (Conceptual): Why does HumanInTheLoopMiddleware require an active checkpointer to work?

What would happen if an agent encountered an interrupt without a checkpointer configured?

> **Answer:** An interrupt suspends Python execution mid-graph. To allow the process to exit, return an HTTP response, or wait hours for a human review, the entire execution state (active node, call stack, message history) must be serialized and saved to durable storage. Without a checkpointer, the state would vanish from RAM upon function return.

Question 2 (Code Tracing): In Cell 4, why does the message count drop from 10 to 6 instead of 4?

> **Answer:** `keep=("messages", 4)` dictates how many *existing* messages to preserve alongside the new summary. When the trigger fired on Turn 6:
>
> - 1 synthetic summary message is injected: `HumanMessage(lc_source='summarization')`.
> - 4 recent messages are retained from history (Turns 4 & 5).
> - 1 new model response message is generated for the Turn 6 question.
>
> Total messages = $1 + 4 + 1 = 6$ messages.

Question 3 (Operational): How does the 'edit' decision prevent hallucinated tool parameters?

> **Answer:** When an LLM generates a tool call with slight inaccuracies (e.g., misspelled email `vrad0404@gmail.com` or wrong SQL table name), passing `{"type": "edit", "edited_action": {...}}` replaces the tool arguments *in-flight*. The tool executes immediately with the verified human input without requiring another expensive or unreliable LLM re-generation turn.

Question 4 (Architecture): What is the key advantage of token-based triggers over message-based triggers in tool-heavy agents?

> **Answer:** Tool outputs vary wildly in size: a single database or web search tool result can return 2,000 tokens in one message, whereas a simple greeting returns 5 tokens. Message-count triggers treat both identically, risking severe context overflow on data-dense outputs. Token-based triggers monitor actual byte and token volume, providing deterministic protection against rate limits.

L. Further Learning & Official Documentation

- LangChain Agent Middleware Guide
- LangGraph Human-In-The-Loop Documentation
- LangGraph Persistence & Checkpointers
- Previous study guides in sequence: **4-messages.pdf** and **5-structure.pdf**.